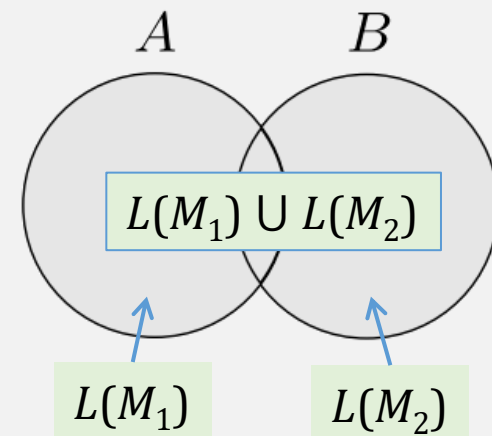


CS 420

Are Regular Languages Closed Under Union?

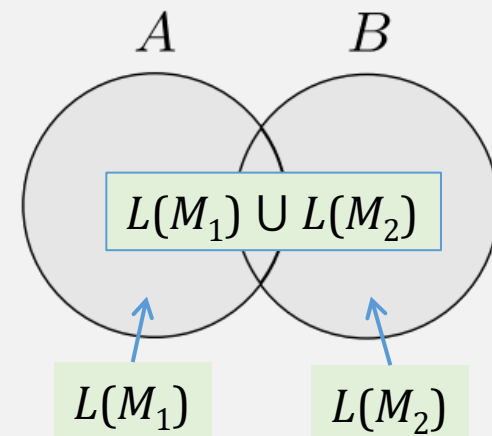
Tuesday, September 29, 2026

UMass Boston Computer Science



Announcements

- HW 1
 - Grades released
- Questions / Complaints about grading?
 - GradeScope re-grade requests welcome
 - But ... this is the only way to discuss grading
 - Do not ask instructor or TAs (we have many graders)
 - Must refer to specific deduction
- HW 2
 - ~~Due: Tue 9/28~~
- HW 3
 - Out: Tue 9/29
 - Due: Tue 10/6 12:30pm



HW Hints

Correct answers ...

- ... use notation from this (semester's) course only
- ... use terminology from this (semester's) course only
- ... are precise, precise, precise
 - E.g., if a question asks for a “set of strings”, then a submitted answer should be ...
- ... do not have extraneous:
 - characters
 - words
 - comments
 - punctuation
 - (don't submit the hw question itself)

These make
answers
incorrect!

Union: $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$

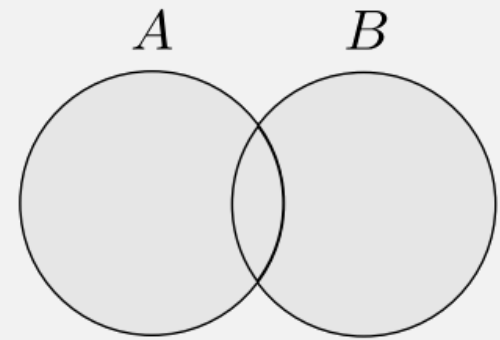
(infix function notation)

Union of Languages

Let the alphabet Σ be the standard 26 letters $\{a, b, \dots, z\}$.

If $A = \{\text{fort, south}\}$ $B = \{\text{point, boston}\}$

$$A \cup B = \{\text{fort, south, point, boston}\}$$



Want: “Closed” Ops on Regular Langs!

- Set of Regular Languages = $\{L_1, L_2, \dots\}$

- Closed under ...?

- Union (“OR”)
- Intersection (“AND”)
- ...

A set is **closed** under an operation if:
the result of applying the operation to
members of the set is in the same set

i.e., input set(s) = output set

???

For Example:

???

$U : \text{Regular Lang} \times \text{Regular Lang} \rightarrow \text{Regular Lang}$

Why Care About Closed Ops on Reg Langs?

- Closed operations for regular langs preserve “regularness”
- I.e., it preserves the same computation model!
- Can “combine” smaller “regular” computations to get bigger ones:

For Example:

$$U : \text{Regular Lang} \times \text{Regular Lang} \rightarrow \text{Regular Lang}$$


- This semester, we will spend a lot of time looking for **operations that are closed!** (for different sets of languages)

Are Regular Languages Closed Under Union?

In this course, we are interested in closed operations for a set of languages (here the set of regular languages)

(In general, a set is **closed** under an operation if applying the operation to members of the set produces a result in the same set)

The class of regular languages is closed under the union operation.

Want to prove this statement

In other words, IF A_1 IS-A Regular Language AND A_2 IS-A Regular Language THEN $A_1 \cup A_2$ IS-A Regular Language

Or this (same) statement

Are Regular Languages Closed Under Union?

THEOREM

The class of regular languages is closed under the union operation.

In other words, IF A_1 IS-A Regular Language AND A_2 IS-A Regular Language
THEN $A_1 \cup A_2$ IS-A Regular Language

(In general, a set is closed under an operation if applying the operation to members of the set produces a result in the same set)

A member of the set of regular languages is ...

... a regular language, which itself is a set (of strings) ...

???

i.e.,

???

$U : \text{Regular Lang} \times \text{Regular Lang} \rightarrow \text{Regular Lang}$

... so the operations we're interested in are set operations

Want to prove this statement

Or this (same) statement

Are Regular Languages Closed Under Union?

THEOREM

The class of regular languages is closed under the union operation.

In other words,

Want to prove this statement

IF A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language
THEN $A_1 \cup A_2$ IS-A **Regular** Language

How to prove an IF-THEN statement?

Proof constructor?

Is Union Closed For Regular Langs?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
THEN $A_1 \cup A_2$ IS-A **Regular** Language

“assume the IF part ...”

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. M_1 IS-A **DFA** Machine AND
 M_1 **recognizes** A_1 ???

Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLang TO ???

???

DefRegLang

IF M IS-A **DFA** Machine AND M **recognizes** L
THEN L IS-A **Regular** Language

7. $A_1 \cup A_2$ IS-A **Regular** Language

“... prove the THEN part”

IF A THEN B == “IF B THEN A ” ??

Statements

1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. M_1 IS-A **DFA** Machine AND M_1 **recognizes** A_1 ???

Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLang TO Proof₁ ???

DefRegLang

IF M IS-A **DFA** Machine AND M **recognizes** L
THEN L IS-A **Regular** Language



DefRegLang???



IF L IS-A **Regular** Language
THEN M IS-A **DFA** Machine AND M **recognizes** L



Equivalence of Conditional Statements

- Yes or No? “If X then Y ” is equivalent to:
 - “If Y then X ” (**converse**)
 - No!

If Regular, Then has DFA?

IF M IS-A **DFA Machine** AND M **recognizes** L
THEN L IS-A **Regular Language**

- Prove: IF L IS-A **Regular Language** THEN M IS-A **DFA Machine** AND M **recognizes** L

How many different ways are there to prove this stmt??

- Proof (Sketch)

Case analysis:

- Look at all If-then statements (in “library”) of the form:

- “IF ... language L , THEN L IS-A **Regular Language**”

- (At least one is true!) Bc there's no constructor for Reg langs! only way to prove “IS-A RegLang” is with **APPLY**

- Figure out which one(s) led to conclusion:

- “ L IS-A **Regular Language**”

- (There's only 1 way!)

- So it must be that:

IF L IS-A **Regular Language**
THEN M IS-A **DFA Machine** AND M **recognizes** L

DefRegLangCoro

“Corollary”

(because there was only 1 possible way to prove stmt: L IS-A **Regular Language**)

Is Union Closed For Regular Langs?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
THEN $A_1 \cup A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. M_1 IS-A **DFA** Machine AND
 M_1 recognizes A_1 ???

Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLang TO Proof₁

???

DefRegLang

IF M IS-A **DFA** Machine AND M recognizes L
THEN L IS-A **Regular** Language



DefRegLangCoro

7. $A_1 \cup A_2$ IS-A **Regular** Language

IF L IS-A **Regular** Language
THEN M IS-A **DFA** Machine AND M recognizes L

Is Union Closed For Regular Langs?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
THEN $A_1 \cup A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. M_1 IS-A **DFA** Machine AND
 M_1 **recognizes** A_1
4. M_2 IS-A **DFA** Machine AND
 M_2 **recognizes** A_2
5. let $M =$ (TODO: construct the DFA!)
IS-A DFA Machine
6. **M recognizes $A_1 \cup A_2$**
7. **$A_1 \cup A_2$ IS-A **Regular** Language**

How to prove this stmt?

Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLangCoro
TO Proof₁
4. APPLY DefRegLangCoro
TO Proof₂

???? DefRegLang
IF M IS-A **DFA** Machine AND M **recognizes** L
THEN L IS-A **Regular** Language

- ???? DefRegLangCoro
7. IF L IS-A **Regular** Language
THEN M IS-A **DFA** Machine AND M **recognizes** L

DEFINITION

A *finite automaton* is a mk-DFA $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set called the *states*,
2. Σ is a finite set called the *alphabet*,
3. $\delta: Q \times \Sigma \rightarrow Q$ is the *transition function*,
4. $q_0 \in Q$ is the *start state*, and
5. $F \subseteq Q$ is the *set of accept states*.

This is the only way to construct DFAs!

Problem: We don't know exactly what M_1 and M_2 are... but we still know...

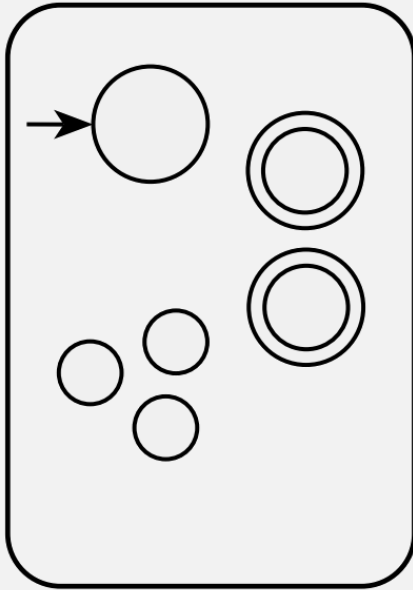
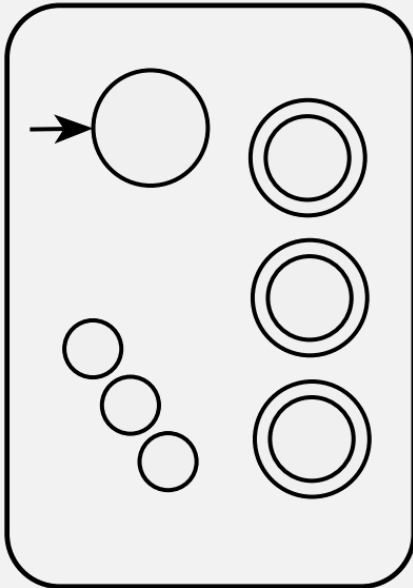
3. M_1 IS-A DFA Machine AND M_1 **recognizes** A_1

4. M_2 IS-A DFA Machine AND M_2 **recognizes** A_2

5. let $M =$ (TODO: construct the DFA!)
IS-A **DFA Machine**

$M_1 = \text{mk-DFA}(Q_1, \Sigma, \delta_1, q_1, F_1)$ for some Q_1 , etc ...

$M_2 = \text{mk-DFA}(Q_2, \Sigma, \delta_2, q_2, F_2)$ for some Q_2 , etc ...

M_1 recognizes A_1  M_2 recognizes A_2 **DEFINITION**

A *finite automaton* is a mk-DFA $(Q, \Sigma, \delta, q_0, F)$, where

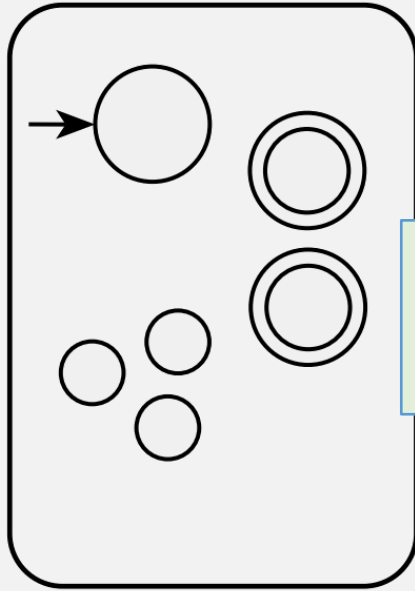
1. Q is a finite set called the *states*,
2. Σ is a finite set called the *alphabet*,
3. $\delta: Q \times \Sigma \rightarrow Q$ is the *transition function*,
4. $q_0 \in Q$ is the *start state*, and
5. $F \subseteq Q$ is the *set of accept states*.

Problem: We don't know exactly what M_1 and M_2 are... but we still know...

$M_1 = \text{mk-DFA}(Q_1, \Sigma, \delta_1, q_1, F_1)$ for some Q_1 , etc ...

$M_2 = \text{mk-DFA}(Q_2, \Sigma, \delta_2, q_2, F_2)$ for some Q_2 , etc ...

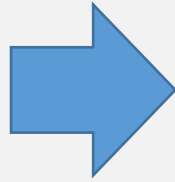
M_1
recognizes A_1



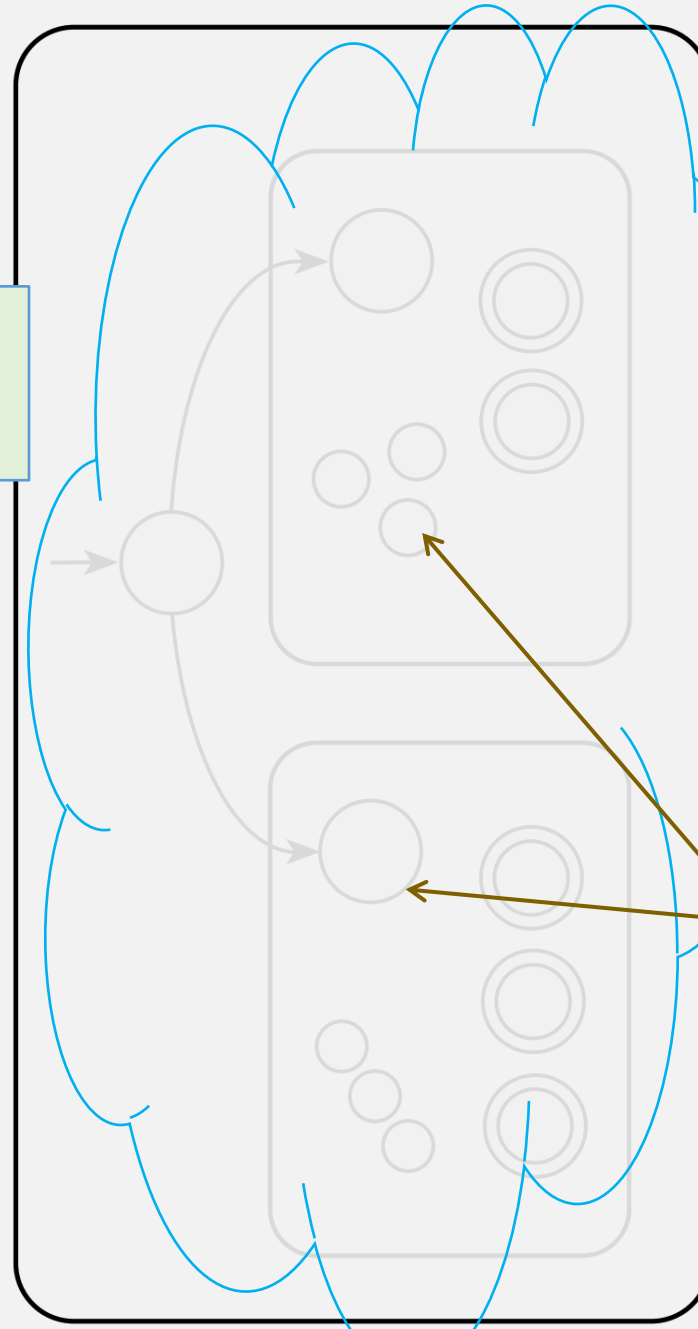
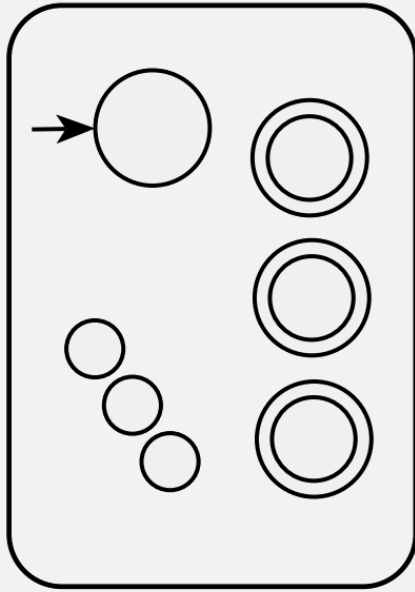
Want: M

Recognizes
 $A_1 \cup A_2$

(to prove $A_1 \cup A_2$
IS-A **Regular**
Language)



M_2
recognizes A_2



Union

Rough sketch Idea:
 M is a combination
of M_1 and M_2 that:
checks whether its
input is accepted
by either M_1 or M_2

But: a DFA can only
read its input once!

Need to: somehow
simulate “being in”
both an M_1 and M_2
state simultaneously

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
THEN $A_1 \cup A_2$ IS-A **Regular** Language

Are Regular Languages Closed Under Union?

Proof (continuation)

Define function $\text{UNION}_{\text{DFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{DFA}$

$\text{UNION}_{\text{DFA}}(M_1, M_2) = M$, where

- Input: $M_1 = \text{mk-DFA}(Q_1, \Sigma, \delta_1, q_1, F_1)$ for some Q_1 , etc.
 $M_2 = \text{mk-DFA}(Q_2, \Sigma, \delta_2, q_2, F_2)$ for some Q_2 , etc.

Want: M that can simultaneously “be in” both an M_1 and M_2 state
- Construct: $M = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$ using M_1, M_2 parts (that accepts if either accepts)
 where:
- states of M : $Q = \{(r_1, r_2) \mid r_1 \in Q_1 \text{ and } r_2 \in Q_2\} = Q_1 \times Q_2$
 This set is the *Cartesian product* of sets Q_1 and Q_2

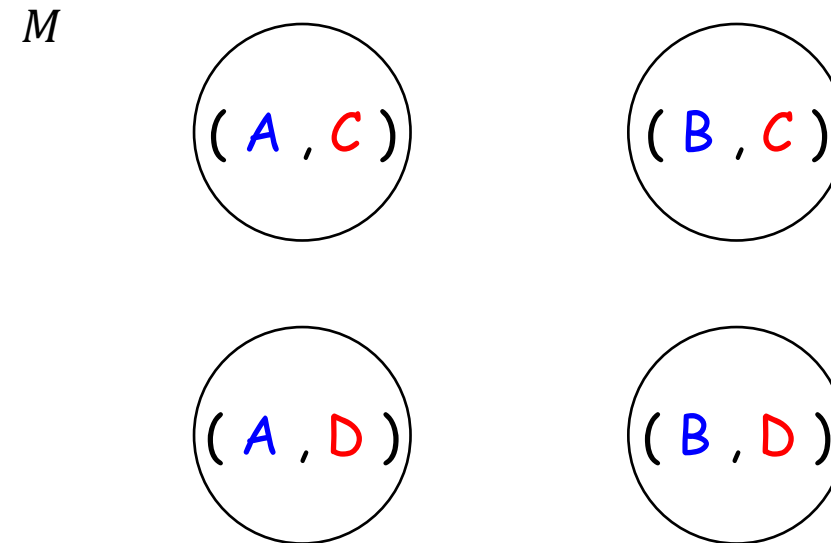
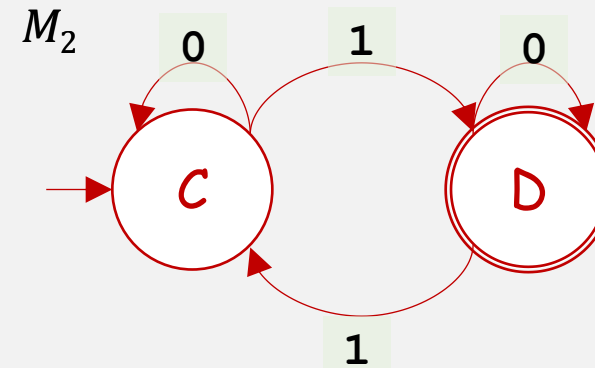
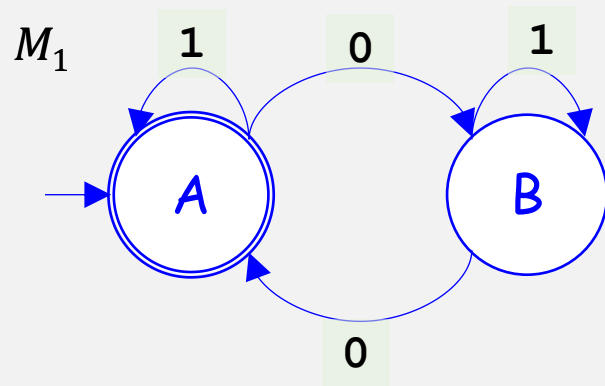
A *finite automaton* is a $\text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set called the *states*,
2. Σ is a finite set called the *alphabet*,
3. $\delta: Q \times \Sigma \rightarrow Q$ is the *transition function*,¹
4. $q_0 \in Q$ is the *start state*, and
5. $F \subseteq Q$ is the *set of accept states*.

A state of M is a pair:
- first part: state of M_1
- second part: state of M_2

states of M : all possible pair combinations of states of M_1 and M_2

UNION_{DFA} Example



Note:

We do not know M_1 or M_2 exactly!

But: a concrete example helps understanding

states of M : all possible pair combinations of states of M_1 and M_2

Are Regular Languages Closed Under Union?

Proof (continuation)

Define function $\text{UNION}_{\text{DFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{DFA}$

$\text{UNION}_{\text{DFA}}(M_1, M_2) = M$, where

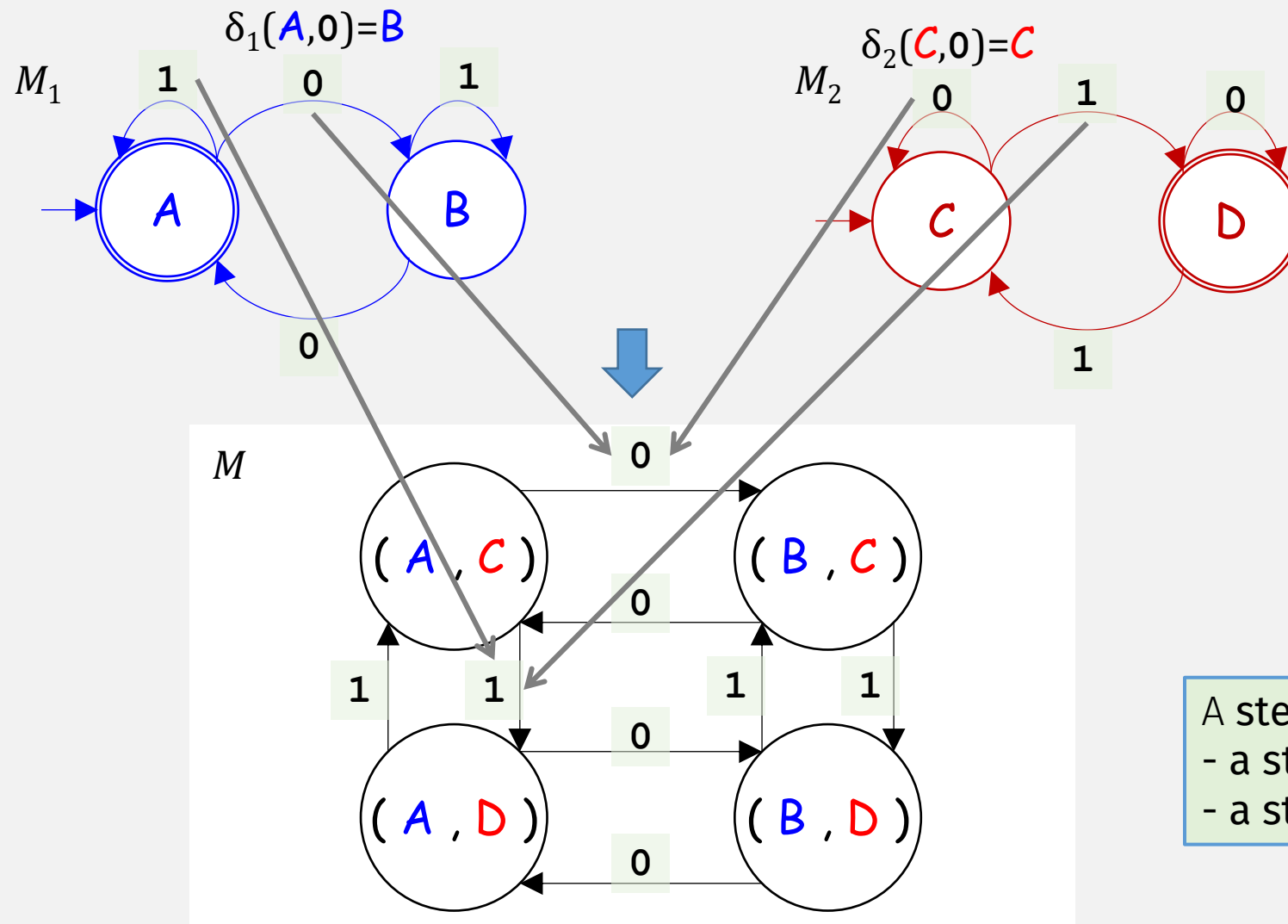
- Input: $M_1 = \text{mk-DFA}(Q_1, \Sigma, \delta_1, q_1, F_1)$
 $M_2 = \text{mk-DFA}(Q_2, \Sigma, \delta_2, q_2, F_2)$
- Construct: $M = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$ using M_1, M_2 parts (that accepts if either accepts)
where:
 - states of M : $Q = \{(r_1, r_2) \mid r_1 \in Q_1 \text{ and } r_2 \in Q_2\} = Q_1 \times Q_2$
This set is the *Cartesian product* of sets Q_1 and Q_2

A *finite automaton* is a $\text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set called the *states*,
2. Σ is a finite set called the *alphabet*,
3. $\delta: Q \times \Sigma \rightarrow Q$ is the *transition function*,
4. $q_0 \in Q$ is the *start state*, and
5. $F \subseteq Q$ is the *set of accept states*.

$$\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$$

A step in M is both:
- a step in M_1 , and
- a step in M_2



A step in M is both:

- a step in M_1 , and
- a step in M_2

Are Regular Languages Closed Under Union?

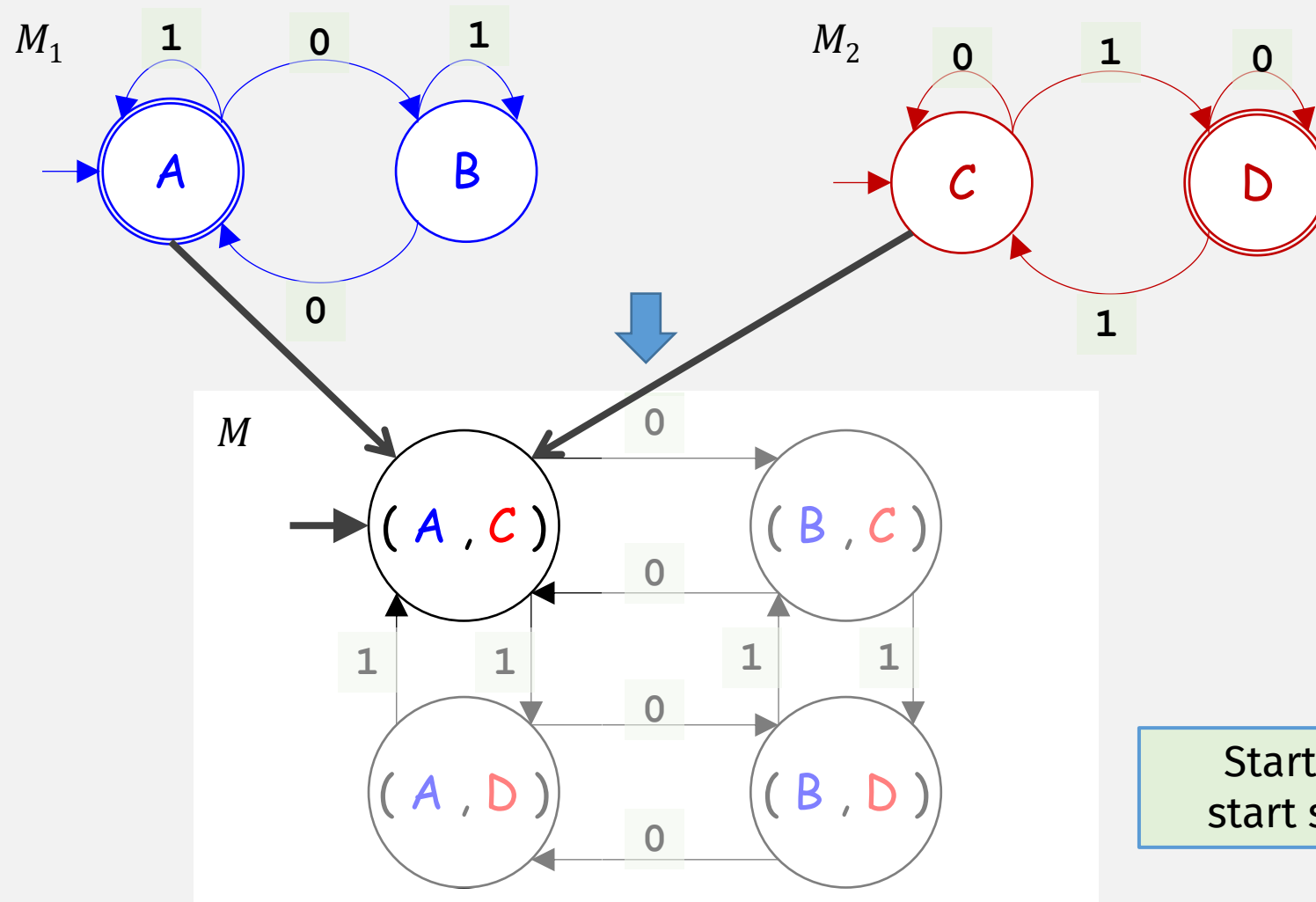
Proof (continuation)

Define function $\text{UNION}_{\text{DFA}} : \mathbf{DFA} \times \mathbf{DFA} \rightarrow \mathbf{DFA}$

$\text{UNION}_{\text{DFA}}(M_1, M_2) = M$, where

- Input: $M_1 = \text{mk-DFA}(Q_1, \Sigma, \delta_1, q_1, F_1)$
 $M_2 = \text{mk-DFA}(Q_2, \Sigma, \delta_2, q_2, F_2)$
- Construct: $M = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$ using M_1, M_2 parts (that accepts if either accepts)
where:
- states of M : $Q = \{(r_1, r_2) \mid r_1 \in Q_1 \text{ and } r_2 \in Q_2\} = Q_1 \times Q_2$
This set is the *Cartesian product* of sets Q_1 and Q_2
- M transition fn: $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$
- M start state: $q_0 = (q_1, q_2)$

Start state of M is both
start states of M_1 and M_2



Start state of M is both
start states of M_1 and M_2

Are Regular Languages Closed Under Union?

Proof (continuation)

Define function $\text{UNION}_{\text{DFA}} : \mathbf{DFA} \times \mathbf{DFA} \rightarrow \mathbf{DFA}$

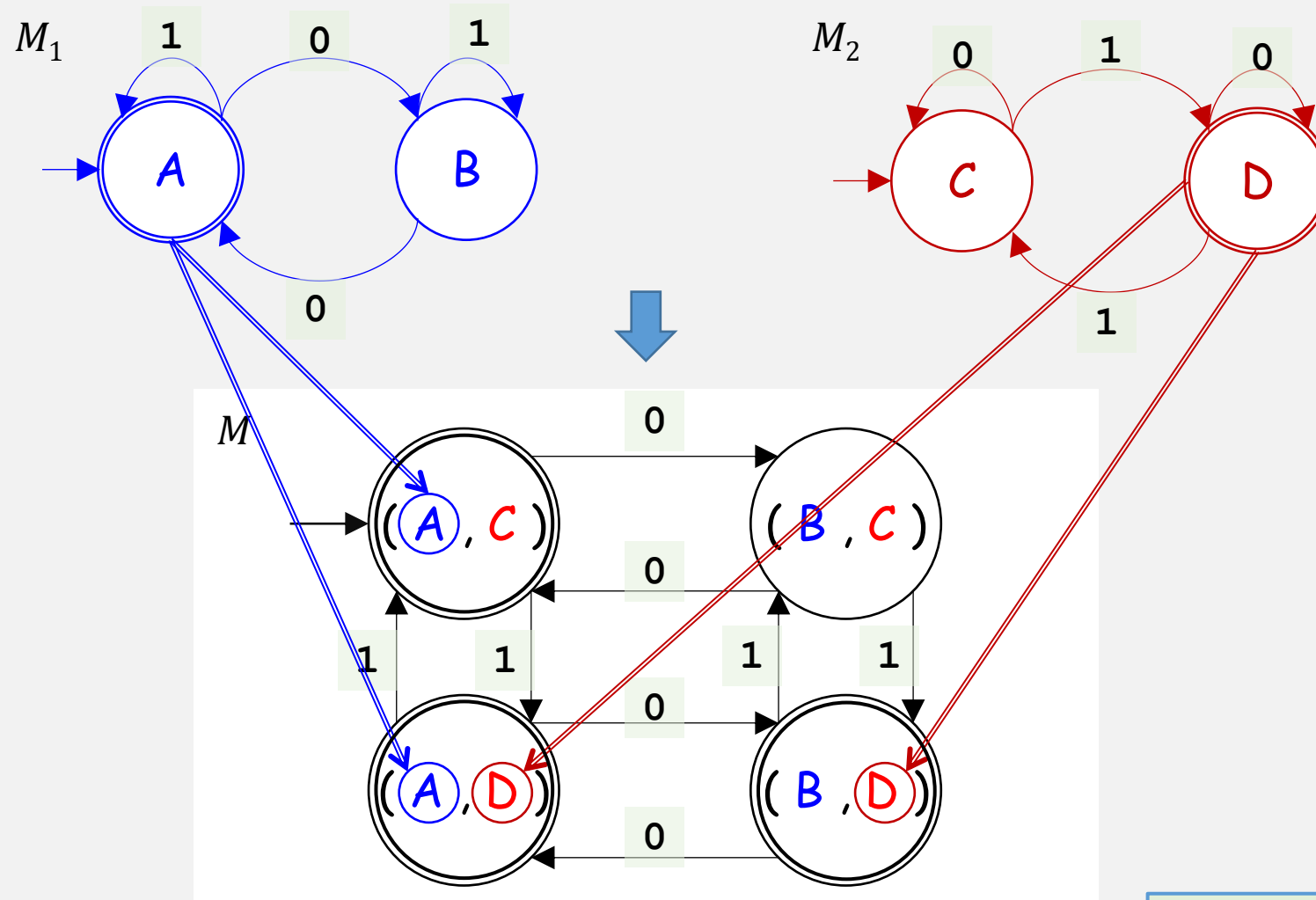
$\text{UNION}_{\text{DFA}}(M_1, M_2) = M$, where

- Input: $M_1 = \text{mk-DFA}(Q_1, \Sigma, \delta_1, q_1, F_1)$
 $M_2 = \text{mk-DFA}(Q_2, \Sigma, \delta_2, q_2, F_2)$
- Construct: $M = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$ using M_1, M_2 parts (that accepts if either accepts)
where:
 - states of M : $Q = \{(r_1, r_2) \mid r_1 \in Q_1 \text{ and } r_2 \in Q_2\} = Q_1 \times Q_2$
This set is the *Cartesian product* of sets Q_1 and Q_2
- M transition fn: $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$
- M start state: $q_0 = (q_1, q_2)$
- M accept states: $F = \{(r_1, r_2) \mid r_1 \in F_1 \text{ or } r_2 \in F_2\}$

Remember:
Accept states must
be subset of Q

Accept if either M_1 or M_2 accept

DFA Union Example



Accept if either M_1 or M_2 accept

Are Regular Languages Closed Under Union?

Proof (continuation)

Define function $\text{UNION}_{\text{DFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{DFA}$

$\text{UNION}_{\text{DFA}}(M_1, M_2) = M$, where

- Input: $M_1 = \text{mk-DFA}(Q_1, \Sigma, \delta_1, q_1, F_1)$
 $M_2 = \text{mk-DFA}(Q_2, \Sigma, \delta_2, q_2, F_2)$
- Construct: $M = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$ using M_1, M_2 parts (that accepts if either accepts)
where:
- states of M : $Q = \{(r_1, r_2) \mid r_1 \in Q_1 \text{ and } r_2 \in Q_2\} = Q_1 \times Q_2$
This set is the *Cartesian product* of sets Q_1 and Q_2
- M transition fn: $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$
- M start state: $q_0 = (q_1, q_2)$
- M accept states: $F = \{(r_1, r_2) \mid r_1 \in F_1 \text{ or } r_2 \in F_2\}$.

Q.E.D.?



Is Union Closed For Regular Langs?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
THEN $A_1 \cup A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. M_1 IS-A **DFA** Machine AND
 M_1 **recognizes** A_1
4. M_2 IS-A **DFA** Machine AND
 M_2 **recognizes** A_2
5. let $M = (\text{TODO: construct the DFA!})$
IS-A **DFA** Machine
6. M **recognizes** $A_1 \cup A_2$
7. $A_1 \cup A_2$ IS-A **Regular** Language

Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLangCoro
TO Proof₁
4. APPLY DefRegLangCoro
TO Proof₂
- ????
- ????
7. APPLY DefRegLang
TO mk-AND-proof Proof₅, Proof₆

Is Union Closed For Regular Langs?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
THEN $A_1 \cup A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. M_1 IS-A **DFA** Machine AND
 M_1 **recognizes** A_1
4. M_2 IS-A **DFA** Machine AND
 M_2 **recognizes** A_2
5. let $M = \text{UNION}_{\text{DFA}}(M_1, M_2)$
IS-A DFA Machine
6. M **recognizes** $A_1 \cup A_2$
7. $A_1 \cup A_2$ IS-A **Regular** Language

Allowed to use let to define **DFA** in an IS-A stmt

Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLangCoro
TO Proof₁
4. APPLY DefRegLangCoro
TO Proof₂

5. APPLY UNION_{DFA}: $\text{DFA} \times \text{DFA} \rightarrow \text{DFA}$
TO $(M_1, M_2) : \text{DFA} \times \text{DFA}$

input

output

????

7. APPLY DefRegLang
TO mk-AND-proof Proof₅, Proof₆

Is Union Closed For Regular Langs?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
THEN $A_1 \cup A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. M_1 IS-A **DFA** Machine AND
 M_1 **recognizes** A_1
4. M_2 IS-A **DFA** Machine AND
 M_2 **recognizes** A_2
5. let $M = \text{UNION}_{\text{DFA}}(M_1, M_2)$
IS-A **DFA** Machine
6. M **recognizes** $A_1 \cup A_2$
7. $A_1 \cup A_2$ IS-A **Regular** Language

Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLangCoro
TO Proof₁
4. APPLY DefRegLangCoro
TO Proof₂
5. APPLY $\text{UNION}_{\text{DFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{DFA}$
TO $(M_1, M_2) : \text{DFA} \times \text{DFA}$
6. BY (TODO: create Example Table!) **COMPARING** $M, A_1 \cup A_2$
7. APPLY DefRegLang
TO mk-AND-proof Proof₅, Proof₆

“Prove” that DFA recognizes a language

let $s_1 \in A_1, s_2 \in A_2$

let $s_3 \notin A_1, s_4 \notin A_2$

Be careful when choosing examples!

In this class, a table like this is sufficient to “prove” that a DFA recognizes a language

let $s =$	$s \in A_1 \cup A_2?$	run M on $s?$
s_1	Yes	
s_2	Yes	
s_3	???	
s_4	???	

Using:

Don't know A_1 and A_2 exactly ...

... but we know ...

... they are **sets of strings!**

$M_1 = \text{mk-DFA}(Q_1, \Sigma, \delta_1, q_1, F_1)$ that **recognizes** A_1

$M_2 = \text{mk-DFA}(Q_2, \Sigma, \delta_2, q_2, F_2)$ that **recognizes** A_2

Constructed:

$M = \text{UNION}_{\text{DFA}}(M_1, M_2) = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$ that **recognizes** $A_1 \cup A_2?$

“Prove” that DFA recognizes a language

let $s_1 \in A_1, s_2 \in A_2$

~~let $s_3 \notin A_1, s_4 \notin A_2$~~

let $s_5 \notin A_1$ and $\notin A_2$

let $s =$	$s \in A_1 \cup A_2?$	run M on $s?$
s_1	Yes	
s_2	Yes	
s_3	???	
s_4	???	
s_5	No	

From:

$M_1 = \text{mk-DFA}(Q_1, \Sigma, \delta_1, q_1, F_1)$ that recognizes A_1

$M_2 = \text{mk-DFA}(Q_2, \Sigma, \delta_2, q_2, F_2)$ that recognizes A_2

Constructed:

$M = \text{UNION}_{\text{DFA}}(M_1, M_2) = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$ that recognizes $A_1 \cup A_2?$

Regular Languages Are Closed Under Union

Proof (continuation)

- Given: $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$, recognize A_1 ,
 $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$, recognize A_2 ,
- Construct: $M = (Q, \Sigma, \delta, q_0, F)$, using M_1 and M_2 , that recognizes $A_1 \cup A_2$
- states of M :
where:
 $Q = \{(r_1, r_2) \mid r_1 \in Q_1 \text{ and } r_2 \in Q_2\} = Q_1 \times Q_2$
This set is the *Cartesian product* of sets Q_1 and Q_2
- M transition fn: $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$
- M start state: $q_0 = (q_1, q_2)$

Accept if either M_1 or M_2 accept
- M accept states: $F = \{(r_1, r_2) \mid r_1 \in F_1 \text{ or } r_2 \in F_2\}$.

“Prove” that DFA recognizes a language

let $s_1 \in A_1, s_2 \in A_2$

let $s_5 \notin A_1, \notin A_2$

(this column needed when machine is not concrete, i.e., can't directly run machine to check if string is accepted)

let $s =$	$s \in A_1 \cup A_2?$	run M on $s?$	run M on s Justification
s_1	Yes	Accept ??	??
s_2	Yes	Accept ??	??
s_3	???		
s_4	???		
s_5	No	Reject ??	??

From:

$M_1 = \text{mk-DFA}(Q_1, \Sigma, \delta_1, q_1, F_1)$ that recognizes A_1

$M_2 = \text{mk-DFA}(Q_2, \Sigma, \delta_2, q_2, F_2)$ that recognizes A_2

Constructed:

$M = \text{UNION}_{\text{DFA}}(M_1, M_2) = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$ that recognizes $A_1 \cup A_2?$

Accept if either M_1 or M_2 accept

where:

$F = \{(r_1, r_2) \mid r_1 \in F_1 \text{ or } r_2 \in F_2\}$.

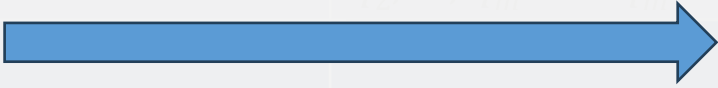
“Prove” that DFA recognizes a language

let $s_1 \in A_1, s_2 \in A_2$

let $s_5 \notin A_1, \notin A_2$

(these extra columns are allowed / required when machine is not concrete, and constructed from smaller machines)

(this column needed when machine is not concrete, i.e., can't directly run machine to check if string is accepted)

let $s =$	$s \in A_1 \cup A_2?$	run M_1 on $s =$	run M_2 on $s =$	run M on $s?$	run M on s Justification
s_1	Yes	$q_1, \dots, q_n$ where $q_n \in F_1$		Accept ??	$(q_1, q_2), \dots, (q_n, _)??$ where $(q_n, _) \in F$
s_2	Yes		$q_1, \dots, q_m$ where $q_m \in F_2$	Accept ??	$(q_1, q_2), \dots, (_, q_m)??$ where $(_, q_m) \in F$
s_3	???			???	
s_4	???			???	
s_5	No	$q_1, \dots, q_n$ where $q_n \notin F_1$	$q_1, \dots, q_m$ where $q_m \notin F_2$	Reject ??	$(q_1, q_2), \dots, (q_n, q_m)??$ where $(q_n, q_m) \notin F$

From:

$M_1 = \text{mk-DFA}(Q_1, \Sigma, \delta_1, q_1, F_1)$ that recognizes A_1

$M_2 = \text{mk-DFA}(Q_2, \Sigma, \delta_2, q_2, F_2)$ that recognizes A_2

Constructed:

where:

$F = \{(r_1, r_2) \mid r_1 \in F_1 \text{ or } r_2 \in F_2\}$.

$M = \text{UNION}_{\text{DFA}}(M_1, M_2) = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$ that recognizes $A_1 \cup A_2?$

let ExTable₁ =

“Prove” that DFA recognizes a language

let $s_1 \in A_1, s_2 \in A_2$

let $s_5 \notin A_1, \notin A_2$

(these extra columns are allowed / required when machine is not concrete, and constructed from smaller machines)

(this column needed when machine is not concrete, i.e., can't directly run machine to check if string is accepted)

let $s =$	$s \in A_1 \cup A_2?$	run M_1 on $s =$	run M_2 on $s =$	run M on $s?$	run M on s Justification
s_1	Yes	$q_1, \dots, q_n$ where $q_n \in F_1$		Accept ??	$(q_1, q_2), \dots, (q_n, _)$?? where $(q_n, _) \in F$
s_2	Yes	A run of a machine = sequence of states	$q_2, \dots, q_m$ where $q_m \in F_2$	Accept ??	$(q_1, q_2), \dots, (_, q_m)$?? where $(_, q_m) \in F$
s_3	???	An accepting run ends in accept state!	A rejecting run <u>does not</u> end in accept state!	???	
s_4	???			???	
s_5	No	$q_1, \dots, q_n$ where $q_r \notin F_1$	$q_2, \dots, q_m$ where $q_s \notin F_2$	Reject ??	$(q_1, q_2), \dots, (q_r, q_s)$?? where $(q_r, q_s) \notin F$

From:

$M_1 = \text{mk-DFA}(Q_1, \Sigma, \delta_1, q_1, F_1)$ that recognizes A_1

$M_2 = \text{mk-DFA}(Q_2, \Sigma, \delta_2, q_2, F_2)$ that recognizes A_2

Constructed:

$M = \text{UNION}_{\text{DFA}}(M_1, M_2) = \text{mk-DFA}(Q, \Sigma, \delta, q_0, F)$ that recognizes $A_1 \cup A_2?$

Accept if either M_1 or M_2 accept

$$F = \{(r_1, r_2) \mid r_1 \in F_1 \text{ or } r_2 \in F_2\}.$$

Previously

Is Union Closed For Regular Langs?, i.e.,

IF A_1 IS-A **Regular** Language AND
 A_2 IS-A **Regular** Language
THEN $A_1 \cup A_2$ IS-A **Regular** Language

mk-IFTHEN-Proof (with ASSUMPTION = proof of A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language):

Statements

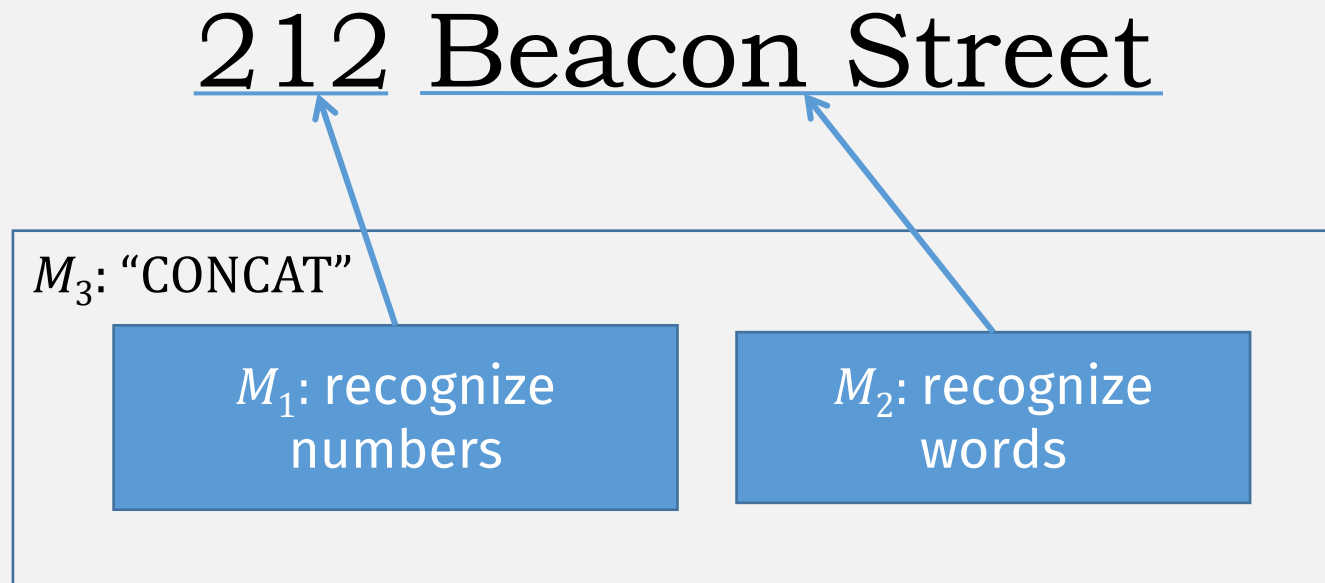
1. A_1 IS-A **Regular** Language
2. A_2 IS-A **Regular** Language
3. M_1 IS-A **DFA** Machine AND
 M_1 **recognizes** A_1
4. M_2 IS-A **DFA** Machine AND
 M_2 **recognizes** A_2
5. let $M = \text{UNION}_{\text{DFA}}(M_1, M_2)$
IS-A **DFA** Machine
6. M **recognizes** $A_1 \cup A_2$
7. $A_1 \cup A_2$ IS-A **Regular** Language

Justifications

1. FIRST ASSUMPTION
2. SECOND ASSUMPTION
3. APPLY DefRegLangCoro
TO Proof₁
4. APPLY DefRegLangCoro
TO Proof₂
5. APPLY $\text{UNION}_{\text{DFA}} : \text{DFA} \times \text{DFA} \rightarrow \text{DFA}$
TO $M_1 : \text{DFA}, M_2 : \text{DFA}$
6. BY (TODO: create Example Table!) **COMPARING** $M, A_1 \cup A_2$
7. APPLY DefRegLang
TO mk-AND-proof Proof₅, Proof₆

Another (common string) operation: Concatenation

Example: Recognizing street addresses



Concatenation: $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$

(infix function notation)

Concatenation of Languages

Let the alphabet Σ be the standard 26 letters $\{a, b, \dots, z\}$.

If $A = \{\text{fort}, \text{south}\}$ $B = \{\text{point}, \text{boston}\}$

$$A \circ B = \{\text{fortpoint}, \text{fortboston}, \text{southpoint}, \text{southboston}\}$$

Regular Languages Closed Under Concat?

THEOREM

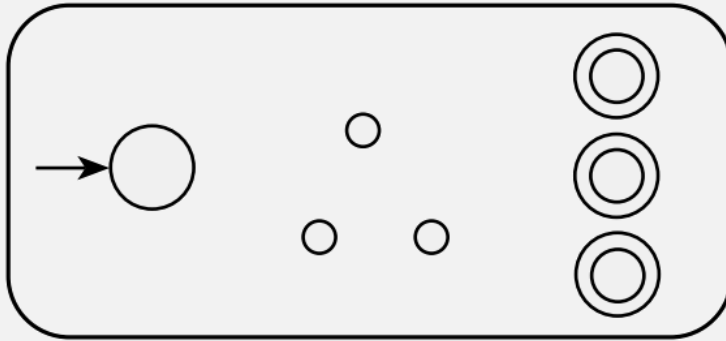
The class of regular languages is closed under the concatenation operation.

In other words, IF A_1 IS-A **Regular** Language AND A_2 IS-A **Regular** Language
THEN $A_1 \circ A_2$ IS-A **Regular** Language

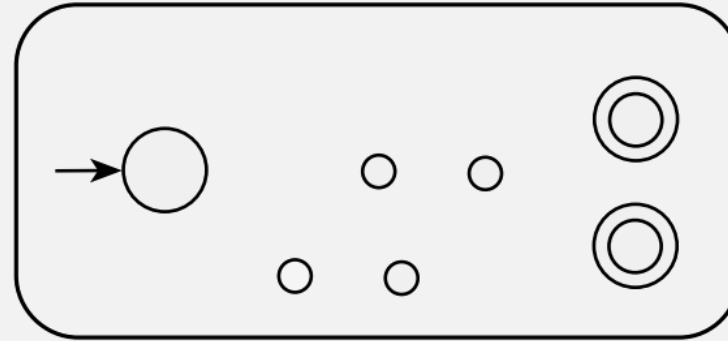
- Construct a new machine M recognizing $A_1 \circ A_2$? (like union)
 - Using DFA M_1 (which recognizes A_1),
 - and DFA M_2 (which recognizes A_2)

Concatentation

M_1



M_2



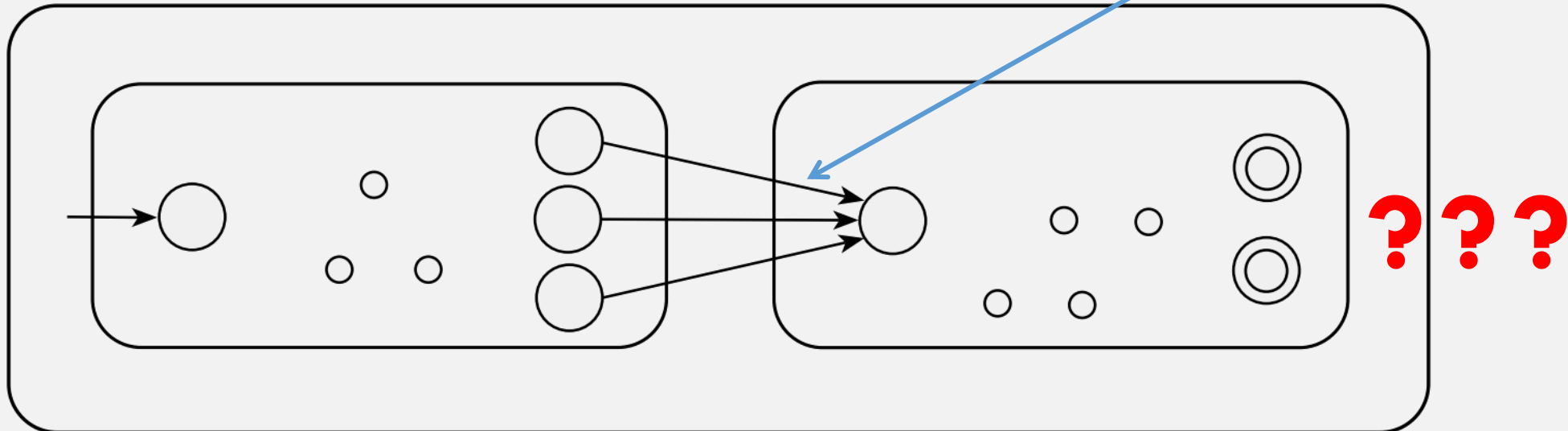
PROBLEM:

Can only read input once, can't backtrack

Let M_1 recognize A_1 , and M_2 recognize A_2 .

Want: Construction of M to recognize $A_1 \circ A_2$

Need to switch machines at some point, but when?



Concatenation: $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$

Overlapping Concatenation Example

- Let M_1 recognize language $A = \{ \text{jen}, \text{jens} \}$
- and M_2 recognize language $B = \{ \text{smith} \}$
- Want: Construct M to recognize $A \circ B = \{ \boxed{\text{jen}}\text{smith}, \boxed{\text{jens}}\text{smith} \}$
- If M sees **jen** ...
- M must decide to either:

Concatenation: $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$

Overlapping Concatenation Example

- Let M_1 recognize language $A = \{ \text{j en, j ens} \}$
- and M_2 recognize language $B = \{ \text{smith} \}$
- Want: Construct M to recognize $A \circ B = \{ \text{j ensmith, j enssmith} \}$
- If M sees **j en** ...
- M must decide to either:
 - stay in M_1 (correct, if full input is **j en****smith**)

Concatenation: $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$

Overlapping Concatenation Example

- Let M_1 recognize language $A = \{ \text{jen}, \text{jens} \}$
- and M_2 recognize language $B = \{ \text{smith} \}$
- Want: Construct M to recognize $A \circ B = \{ \text{jensmith}, \text{jenssmith} \}$

- If M sees jen ...
- M must decide to either:
 - stay in M_1 (correct, if full input is jenssmith)
 - or switch to M_2 (correct, if full input is jenssmith)
- But to recognize $A \circ B$, it needs to handle both cases!!
 - Without backtracking

A DFA can't do this!

Regular Languages Closed Under Concat?

FALSE?

THEOREM

The class of regular languages is closed under the concatenation operation.

- Cannot combine A_1 and A_2 's machine because:
 - Need to switch from A_1 to A_2 at some point ...
 - ... but we don't know when! (we can only read input once)
- This requires a new kind of machine!
- But does this mean concatenation is not closed for regular langs?